AIStudio

Quickstart

Version: Beta | Updated: 2026-07-12

Get a running AIStudio instance in under 30 minutes.

AIStudio runs entirely on your Mac — no cloud account, no API keys, no data leaving your machine. You'll have a local AI search engine over your own documents, accessible from your browser.

> ***This guide is intentionally detailed*** *— it explains what each tool does and why it is needed. This is deliberate: AIStudio has several components, and understanding what you are installing makes you a much more effective user.* ***In a hurry, and already have the developer tools?*** *Experienced users who already run Homebrew, Python, Ollama, and git can skip to the* ***TL;DR — Fast Install*** *at the end and be running in minutes — but you'll miss the explanations of how a RAG system is built (the fun part).*

A Note on the Tools Used in This Guide

Installing AIStudio requires a few external tools — package managers, a model runtime, a vector database, and a code host. Each is introduced where it is first needed; for a one-glance summary of all of them with links, see **Annex 1 — Tools Used in This Guide**.

The two you'll lean on most are both places software is stored and downloaded from automatically:

Homebrew (https://brew.sh) is a package manager for macOS. Think of it as an App Store for developer tools — instead of clicking buttons in a GUI, you type `brew install <something>` and Homebrew finds it, downloads it, and installs it for you. We use it to install Python and Ollama.

GitHub (https://github.com) is where AIStudio's code lives. Think of it as a library where software is stored and versioned. We use a tool called `git` to download ("clone") AIStudio from GitHub directly onto your Mac. AIStudio is a public repository — no account or access key required to download it.

> *If a command ever does something unexpected, see **Annex 2 — Troubleshooting** at the end — its entries follow the same order as the steps below.*

Before You Start

Open Terminal first. Press **⌘ Space** (that's the Command key and the Space bar at the same time), type **Terminal**, press **Enter**.

You'll see a window with a prompt that looks something like this:

```
yourusername@Mac ~ %
```

The prompt text varies by machine and depends on your name — don't worry about it.

AIStudio setup uses the shell to get installed and also to perform tasks the AIStudio User Interface (UI), which runs in your browser, doesn't cover. The commands all start with `ais_` — like `ais_start`, `ais_stop`, `ais_bench`. These are presented in Step 8.

> ***What is a shell?** Terminal gives you access to the shell — a text interface for running commands on your Mac.*

If you close the Terminal window at any point, open a new one: press **⌘ Space** (Command key + Space bar), type **Terminal**, press **Enter**. Then re-run the last command you were on and continue from there.

Prerequisites

You need a Mac with Apple Silicon (M1/M2/M3/M4). Everything else you need — Python, Homebrew, Qdrant, Ollama — is installed in the steps below.

> *Not sure if you have Apple Silicon? Click the **Apple menu** (on the top left part of the screen) → **About This Mac**. Look for "Chip: Apple M1" (or M2/M3/M4). Intel Macs have not been tested — results may vary.*

---

0. Confirm Your Shell is zsh

Every command in this guide assumes your Mac's Terminal uses **zsh** — the default shell on macOS since 2019. A few accounts (especially ones created long ago, or set up by hand) still default to the older **bash** shell. On those, this setup *silently half-works*: software installs correctly but the lines that are supposed to make commands permanently available go into the wrong place, so commands you just installed come back as `command not found`. This one check prevents the most common reason the install "looks done but doesn't work."

The quickest tell is your prompt. Look at the very end of the prompt line in Terminal:

- Ends in `%` (for example `yourname@Mac ~ %`) — you're on zsh. → **Skip to Step 1.**
- Ends in `$` (for example `Mac:~ yourname$`) — you're on bash. Switch to zsh (below).

To be certain, type:

```
echo $SHELL
```

- If it prints something ending in `/zsh` (e.g. `/bin/zsh`) — you're set. → **Go to Step 1.**
- If it prints something ending in `/bash` (e.g. `/bin/bash`) — switch your account to zsh:

```
chsh -s /bin/zsh
```

Then **quit Terminal completely** — press **⌘ Q** (Command key + Q) with Terminal in front, *not* just the red close button — and **reopen it** (**⌘ Space**, type **Terminal**, **Enter**). Run `echo $SHELL` once more to confirm it now ends in `/zsh`, then continue with Step 1.

> ***Why this matters:*** *the later steps save settings into files named `~/.zprofile` and `~/.zshrc`. Those files are read by zsh. A bash session ignores them — so on bash the Homebrew path, the Qdrant path, and the `ais_*` commands all appear to install but won't be found in new Terminal windows. Switching to zsh once, here, makes every step below work exactly as written.*

---

1. Install Homebrew

Homebrew is a package manager for macOS — it installs the software AIStudio needs.

First, check if it is already installed; in the Terminal, type:

```
brew --version
```

If you see a version number — Homebrew is already installed. → **Skip to Step 2.**

If you see `command not found`, install Homebrew. You will be asked for your Mac password — this is the same password you use to unlock your Mac. Type it and press Enter. Nothing will appear on screen as you type — that is normal:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.
```

The installer will show a list of directories it will create. Press **Enter** to continue.

> ***On a brand-new or freshly-wiped Mac, this step installs the developer command-line tools first.*** *Before Homebrew finishes, macOS may pop up a dialog titled* ***"The `xcode-select` command requires the command line developer tools"*** *(or Homebrew prints `xcode-select: note: install requested`).* ***Click "Install" in that dialog and wait for the multi-gigabyte download to finish*** *— it can take several minutes. This is normal when setting up a Mac from scratch, and it is what gives you `git`. You cannot skip it or script past it; let it complete, then continue here.*

When installation completes you will see `==> Installation successful!` Homebrew then prints a few more blocks — an analytics notice (with an opt-out link), a donation note, and a `==> Next steps:` section. **These are all informational — you don't need to act on any of them** (the "Next steps" are generic Homebrew tips, not part of AIStudio setup). Modern Macs set the Homebrew PATH automatically. Verify:

```
brew --version
```

If you see a version number — you are all set. On older Macs, if you still see `command not found`, run these three lines (and see Annex 2 — Troubleshooting if it persists):

```
echo >> ~/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

---

2. Install Python

First, check if Python is already installed; in the Terminal, type:

```
python3 --version
```

If you see a version number where the second part is 10 or higher — for example Python 3.10, 3.11, 3.12, 3.13, or 3.14 — Python is recent enough for AIStudio. → **Skip to Step 3. Do not run the install command below** — it would install an *older* Python (3.13) alongside the newer one you already have.

"3.10 or higher" means: look at the number after the first dot. If it is 10 or more, you are good.

If you see `command not found` **— or a version whose second number is less than 10** (for example `Python 3.9.x`, which ships on many Macs and is too old for AIStudio) — install a current Python via Homebrew:

```
brew install python@3.13
```

Homebrew shows what it will install, then asks `Do you want to proceed with the installation? [y/n]` — type `y` and press **Enter**. (It only proceeds once you answer; it will sit and wait otherwise.) This takes less than a minute. You'll see a stream of messages, including a `==> Caveats` section — that is normal, and you don't need to read it.

You may also see notices like:

```
[notice] A new release of pip is available
```

Ignore these — they do not affect AIStudio.

Homebrew does **not** automatically make this the `python3` your Mac reaches for — the new Python is installed but "parked" off to the side, so `python3 --version` will still report the old built-in 3.9. Point your Mac at the new one (copy and paste the whole block at once; it's safe to run more than once):

```
PY_LINE='export PATH="/opt/homebrew/opt/python@3.13/libexec/bin:$PATH"'
grep -qxF "$PY_LINE" ~/.zprofile 2>/dev/null || echo "$PY_LINE" >> ~/.zprofile
eval "$PY_LINE"
```

Verify:

```
python3 --version
```

Expected: `Python 3.13.x`. If it still shows `3.9.x`, close the Terminal window, open a new one, and run `python3 --version` again.

> *AIStudio uses type syntax (`float | None`) that fails on Python 3.9. The macOS system Python is often 3.9 — that's why we check, install a current one, and put it on your PATH.*

---

3. Install Ollama

Ollama (https://ollama.com) is a local model runtime — it downloads, manages, and serves AI language models on your machine. Think of it as a local equivalent of the OpenAI API, running entirely on your hardware.

Check if already installed; in the Terminal, type:

```
ollama --version
```

If you see a version number — Ollama is installed. → **Skip to "Start Ollama" below.**

If Ollama is not installed:

```
brew install ollama
```

When it asks `Do you want to proceed with the installation? [y/n]` , type `y` and press **Enter**. This takes less than a minute. You'll see a stream of messages — ignore them. All is good when you're back at the `%` prompt.

> *You may see a notification saying **"Background Items Added — ollama is an item that can run in the background."** Click to dismiss or ignore it. This means Ollama will start automatically when your Mac starts up — which is exactly what you want.*
>
> *You may also see technical notes about `OLLAMA_FLASH_ATTENTION` and background services — ignore them. Ollama is installed correctly.*

Start Ollama. How you do this depends on how Ollama was installed:

- **Installed via Homebrew just now** (`brew install ollama` above) — start it as a background service: `bash brew services start ollama`
- **Installed as the Ollama app** (downloaded from ollama.com), or it was already on your Mac — it already runs automatically as a background item. **Do not run** `brew services start ollama` — for an app install it errors with `Error: Formula` ollama `is not installed.` There's nothing to start; skip straight to the verify step.

> *The real check that Ollama is working is `ollama list` (below), **not** the `brew services` command — `ollama list` works for both install types.*

Verify Ollama is running:

```
ollama list
```

If you see the column headers (NAME, ID, SIZE, MODIFIED) — Ollama is running correctly. The list may be empty — that is normal. You will download models next.

If you see an error, see Annex 2 — Troubleshooting, or start Ollama manually in a separate terminal tab:

```
ollama serve
```

Install Pango and pandoc (for PDF reports)

The benchmark PDF reports (Module 5 of the Tutorial) need two Homebrew tools: **Pango** (a text-rendering library) and **pandoc** (a document converter). Install both:

```
brew install pango pandoc
```

When it asks `Do you want to proceed with the installation? [y/n]`, type `y` and press **Enter**.

> ***Why these two?** AIStudio's benchmark runner builds each PDF in two steps: `pandoc` converts the Markdown report to HTML, then `weasyprint` (which depends on Pango) renders the HTML to PDF. Missing either one just skips the PDF — the `.md` and `.json` reports are still written — but installing both now means Module 5 produces the full set. `brew install` handles them in under a minute.*

---

4. Pull Required Models

AIStudio uses two kinds of model that do completely different jobs.

nomic-embed-text is the indexing model. When you upload a document, this model reads it and converts every passage into a set of numbers that capture its meaning — called an "embedding." When you ask a question, it finds the passages whose meaning is closest to your question. It is small (274 MB), fast, and runs invisibly. You will never interact with it directly. **Everyone needs this model.**

The language model reads the relevant passages found by the indexing model and writes a human-language answer with citations. This is what most people think of as "AI." AIStudio supports three open model families, each in a small (fast) and a large (highest-quality) size:

Family	Small — fast	Medium — balanced	Large — best quality
Meta Llama ☆	`llama3.1:8b`	—	`llama3.1:70b`
Google Gemma ★	`gemma3:4b`	`gemma3:12b`	`gemma3:27b`
Mistral AI	`mistral:7b`	—	`mixtral:8x7b`

☆ **default** (`llama3.1:8b`) — the fast first-impression model. ★ **benchmark-normalized** (`gemma3:27b`) — the model AIStudio's published SEC/ESEF evidence suite runs on, and the one you switch to for those corpora.

The default is `llama3.1:8b` — Meta's Llama. It gives clean, well-grounded answers on the demo and help corpora out of the box and generally follows the citation format that keeps answers verifiable. Citation *rendering* can vary with your machine and retrieval load — see the TUTORIAL §1.2 note for when it fluctuates and the simple settings (lower Top K, the RAM-appropriate model) that keep it reliable. **`gemma3:27b` is the model behind AIStudio's top benchmark results** — it binds citations cleanly and is what you'll switch to for the heavier SEC and ESEF corpora; **`gemma3:12b` (~8 GB)** is the natural middle pick for 24–32 GB machines, and `gemma3:4b` is a lighter, faster alternative if you're tight on memory. You can switch models at any time in the UI.

Which to download — based on your Mac's memory (Apple menu → **About This Mac** → "Memory"):

- **8–16 GB** — `llama3.1:8b` (the default). Comfortable on 16 GB; on 8 GB it runs but is tight — `gemma3:4b` is a lighter, faster fallback there if you want more headroom.
- **24–32 GB** — `gemma3:12b` (~8 GB) is the sweet spot: most of the quality of the 27B at a fraction of the footprint. On a solid 32 GB machine you can also pull `gemma3:27b` for top-quality answers on heavier corpora.
- **64 GB or more** — `gemma3:27b` is excellent; pull `llama3.1:70b` or `mixtral:8x7b` only if you want the largest Meta / Mistral options.

> ***Don't run a "large" model on too little memory.*** *`gemma3:27b` and `mixtral:8x7b` want 32 GB+; **never** pull `llama3.1:70b` under 64 GB — it will download but won't fit in memory, and your Mac will become unresponsive when you try to use it.*
>
> ***AIStudio guards against this for you.*** *The model picker checks each model against your machine's available memory: one that won't fit is marked and **disabled**, with a one-click recommendation of the largest model that does fit, and a query against a too-large model is refused with a clear message rather than left to silently load and hang. (The same check runs in the API and the benchmark, so a `curl` or `ais_bench` call gets the same protection — see `ais_bench --fit-policy`.)*

Step 1 — Download the indexing model (required for everyone):

```
ollama pull nomic-embed-text
```

Step 2 — Download your language model. Start with the default:

```
ollama pull llama3.1:8b
```

Optional — the larger Gemma for heavier corpora (32 GB+):

```
ollama pull gemma3:27b
```

Prefer a lighter model or another family? Small: `ollama pull gemma3:4b` (lighter, faster) or `ollama pull mistral:7b`. Large: `ollama pull llama3.1:70b` or `ollama pull mixtral:8x7b`.

> ***You'll switch to `gemma3:27b` later — on purpose.*** *The demo corpus runs beautifully on the default `llama3.1:8b`. When you reach the SEC 10-K corpus in the Tutorial, it prompts you to switch to `gemma3:27b` in the model dropdown — the heavier corpus rewards the larger model (it's the model behind AIStudio's top benchmark). Fast first impression now; full power when it counts.*
>
> ***The same rule applies when you benchmark.*** *`ais_bench` (Module 5 of the Tutorial) runs whatever model you point it at. The simplest safe choice on any Mac is **`ais_bench --batch`**, which picks models that fit your machine automatically; **`ais_bench --canonical`** reproduces the published runs on their pinned 27B model and needs 32 GB+. If you name a model yourself and it won't fit, `ais_bench` **offers what does** — a pick-list when several fit, `(r)un / (f)ree memory / (Enter) cancel` when only one does — or `--fit-policy downshift` to choose for you. It never silently hangs your Mac.*

> ***One caveat, because it is easy to get wrong:*** *fitting and sustaining a long run are different things. A model that loads fine can still exhaust memory part-way through a ten-question sweep, since each question's working memory adds up. On a 24 GB Mac we measured* `llama3.1:8b` *completing roughly seven of ten questions and* `gemma3:12b` *about one — both are perfectly usable for asking questions one at a time in the UI, but neither reliably finishes an unattended sweep there. If later questions come back instantly with no citations, that is the memory guard protecting you rather than the model failing: free memory with* `ollama stop <model>` *and run again.*
>
> ***Download times*** *depend on your internet speed (check yours at fast.com). On a 100 Mbps connection, expect about 5 minutes for* `llama3.1:8b` *(or ~2 minutes for the lighter* `gemma3:4b` *) and 10 minutes for* `gemma3:27b` *. The estimate shown during download may fluctuate — starting at hours, then dropping to minutes.*
>
> ***Note:*** *If your models show a modified date of several weeks ago — that's fine. They don't expire.*

---

5. Install Qdrant

Qdrant (https://qdrant.tech) is the database that stores your document chunks. When AIStudio ingests a document, it splits it into overlapping passages — typically a few paragraphs each — and stores them as vectors in Qdrant. When you query, AIStudio searches across all chunks to find the most relevant passages.

> ***Why not Homebrew?*** *Qdrant is a Rust-based binary not available via Homebrew — install it directly.*

Check if already installed; in the Terminal, type:

```
qdrant --version
```

If you see a version number — → **Skip to Step 6.**

If you see `zsh: command not found: qdrant`, Qdrant isn't installed yet — first create a home for its data:

```
mkdir -p ~/qdrant_storage
```

This command returns no output — silence means success.

Download and install the binary, put it on your PATH, and verify:

```
curl -L https://github.com/qdrant/qdrant/releases/latest/download/qdrant-aarch64-apple-darwi
mkdir -p ~/bin
mv qdrant ~/bin/qdrant
grep -q 'export PATH="$HOME/bin' ~/.zshrc 2>/dev/null || echo 'export PATH="$HOME/bin:$PATH'
source ~/.zshrc
qdrant --version
```

> ***Why the `grep` before the `echo`?*** *`~/bin` is where Qdrant now lives, and your PATH is the list of places your Mac searches when you type a command — so `~/bin` has to be on it for `qdrant` to be found. The part before the `||` quietly checks whether that line is already in your `~/.zshrc`; the line is added **only if it's missing**. If you've set AIStudio up before, it's probably already there — so this leaves it alone instead of writing a second identical copy. That's what makes these commands genuinely safe to re-run.*

Expected: `qdrant 1.17.0` (or newer). You will also see:

```
<jemalloc>: option background_thread currently supports pthread only
```

This warning is harmless — ignore it.

---

6. Check You Have git

`git` is the tool that downloads ("clones") AIStudio from GitHub. **If you installed Homebrew in Step 1, you already have it** — Homebrew installs Apple's Command Line Tools (which include `git`) as part of its own setup. This step just confirms it.

Check; in the Terminal, type:

```
git --version
```

If you see `git version 2.x.x` or newer — → **skip to Step 7, Clone AIStudio.** (This is the normal case after Step 1.)

Only if you see `command not found` (you skipped Homebrew, or are on an unusual setup) — install the Command Line Tools directly:

```
xcode-select --install
```

A dialog will appear saying "The xcode-select command requires the command line developer tools. Would you like to install the tools now?" — click **Install**, then **Agree** to the License

Agreement. The progress dialog's time estimate may start very high — even hours — then drop to minutes within seconds; ignore the initial estimate (on a 100 Mbps connection expect about 8–10 minutes). When done, re-run `git --version` to confirm `git version 2.x.x` or newer.

> *If git is already present, running `xcode-select --install` simply reports "Command line tools are already installed" and does nothing — that's expected, not an error. If it genuinely won't install, see Annex 2 — Troubleshooting.*

---

7. Clone AIStudio

AIStudio is a public repository — no GitHub account, SSH key, or access token required. Create a folder for it and clone the repository:

```
mkdir -p ~/Developer
cd ~/Developer
git clone https://github.com/mbarberony/AIStudio.git
cd AIStudio
```

This downloads about 35 MB — usually well under a minute (around 10 seconds on a fast connection). The transfer size shows up in the `Receiving objects:` line as it runs. When complete you will see `Resolving deltas: 100% (...), done.` and land in the `AIStudio` folder.

After the clone, AIStudio lives at `~/Developer/AIStudio` — the path every command in this guide assumes. Your prompt will now end in `AIStudio %` (it shows the current folder's name, not the full path) — that's how you know you're inside the cloned folder.

> ***Cloned to the wrong place?*** *Only if your prompt shows `~/AIStudio` (not `~/Developer/AIStudio`) did the folder land wrong — move it: `bash mkdir -p ~/Developer && mv ~/AIStudio ~/Developer/AIStudio`*

---

8. Install AIStudio Commands

`./ais_install` does two things: it creates the Python environment and installs all dependencies, then makes the `ais_*` commands available in your Terminal.

Run the install:

```
cd ~/Developer/AIStudio
./ais_install
```

When it finishes, load the new commands into your current Terminal, **then** verify — run these on separate lines (don't paste them together; the verify can fire before the load finishes):

```
source ~/.zshrc
```

```
ais_help
```

If `ais_help` returns `command not found`, the install still succeeded — just run `source ~/.zshrc` once more, then `ais_help` again.

The `ais_*` notation is shorthand for *all the commands that begin with* `ais_` — like `ais_start`, `ais_stop`, `ais_bench`; the `*` is a wildcard standing in for the rest of each name. `ais_help` prints the full list by category, and `ais_help <command>` gives detailed help on any one.

> *Most of these commands are exercised in the Tutorial, not here — Modules 2–3 build the SEC 10-K and ESEF corpora (`ais_download_*`, `ais_ingest_*`) and Module 5 runs the benchmark (`ais_bench`). You don't need them yet; the Tutorial introduces each where it's used. For now, only `ais_start` / `ais_stop` matter.*

This takes 2–3 minutes. You'll see progress messages as dependencies install.

> *You may see `WARNING: Cache entry deserialization failed` messages, or a notice like `[notice] A new release of pip is available` — these are harmless. Ignore them.*
>
> *You only need to run `source ~/.zshrc` once — right after installing. Every new Terminal window or tab you open in the future loads your commands automatically.*

---

9. Activate the Virtual Environment (Optional — skip this)

A virtual environment is an isolated Python workspace that keeps AIStudio's dependencies separate from the rest of your system. **You do not need to activate it manually** — `ais_start` handles that for you every time. This step is here only for reference.

You'd run this *only* if you want to use other `ais_*` commands from a fresh terminal tab before `ais_start` has run in that tab:

```
source ~/Developer/AIStudio/.venv/bin/activate
```

This command returns no output — silence means success. Your prompt will show `(.venv)`, confirming the environment is active. (Note: this activates the Python environment but does **not** load the `ais_*` aliases — that's `source ~/.zshrc`.)

---

10. Start AIStudio

```
ais_start
```

AIStudio starts three processes — Qdrant, Ollama, and the FastAPI backend — and opens the UI in your browser automatically, in about 20 seconds.

> *You may see a warning that the backend was slow to start — this is normal on first run. The UI will still open correctly.*
>
> ***First run:** On a fresh install, `ais_start` automatically indexes the **demo** and **help** corpora in the background. A progress banner appears in the UI if indexing takes more than 30 seconds — wait for it to finish before asking your first question.*

It is OK to **minimize** the Terminal window now — but **do not close it**. The Terminal is where the backend runs; closing it shuts AIStudio down.

To verify the backend is up:

```
curl http://localhost:8000/health
```

Expected: `{"status": "ok"}`. If you get an error or no response instead, the backend isn't ready — see Annex 2 — Troubleshooting (start with a hard-refresh of the browser, then `ais_stop` and `ais_start`).

---

11. What You're Looking At — and Your First Query

When AIStudio opens in your browser, you'll see two main areas:

On the left — the control sidebar. Everything that shapes a query lives here: the **corpus selector** at the top (choose which document collection to query — the **demo** corpus is already loaded, original documents spanning 2003–2026 covering enterprise architecture, IT strategy,

financial services, and agentic AI), the **Model** dropdown, and the **query settings** (Top K, Temperature, Timeout, Relevance Threshold, Retrieval Mix). See **Annex 3 — Tuning Parameters** for what each control does.

On the right — the chat area. Type a question, get a cited answer. References below each answer show exactly which document and page the answer came from. Click **Source Dive** ↗ to open the source PDF at the cited page.

The **Model** is set to `llama3.1:8b` by default — small, fast, and reliably cited, ideal for the demo. (For the heavier SEC 10-K corpus in the Tutorial, switch it to `gemma3:27b`.)

Try these questions on the demo corpus to start: - *"What is QFD and how does it apply to technology architecture?"* - *"How do you design an IT organization around architectural principles?"* - *"What does a reference architecture for enterprise AI look like?"*

Then switch to the **help** corpus and ask: *"How do I re-ingest a corpus?"* — AIStudio is answering questions about itself, using the same RAG pipeline to retrieve answers from its own documentation.

> ***Initial latency:*** *Your very first query may take 20–50 seconds while the model loads fully into memory. Every query after that is faster — typically 6–7 seconds.*

---

12. Add a Desktop Shortcut (Optional)

If you minimized the Terminal window in Step 10, reopen it — press **⌘ Space**, type **Terminal**, press **Enter**.

If you'd like to launch AIStudio by double-clicking an icon instead of opening a terminal:

```
ais_create_shortcut
```

This creates `AIStudio.app` in `~/Applications` and adds a shortcut to your Desktop. Double-clicking it starts all AIStudio services and opens the browser automatically — no Terminal needed.

> ***Icon not showing correctly?*** *Run* `killall Dock` *to refresh the icon cache.*

To remove the shortcut later:

```
ais_create_shortcut --remove
```

To skip the Desktop symlink and only create the app in `~/Applications`:

```
ais_create_shortcut --nodesktop
```

Stopping AIStudio

When you're done for the session:

```
ais_stop
```

This shuts down the backend and Qdrant. Your corpora and settings are saved — `ais_start` (or the Desktop shortcut) brings everything back next time.

AIStudio is ready. Your documents are waiting.

★★★ ★★★

For a full guided walkthrough — including the SEC 10-K at-scale exercise and benchmarking — see TUTORIAL.md.

TL;DR — Fast Install

For experienced users who already live in a terminal. This skips every explanation above — you'll get a running instance, but you'll miss the understanding of how a RAG system is built. If anything here is unfamiliar, use the full guide instead.

Prereqs: Apple Silicon Mac, **and a zsh shell** (the macOS default — if your prompt ends in `$` rather than `%`, run `chsh -s /bin/zsh`, quit Terminal with ⌘Q, and reopen before pasting; see Step 0). The steps below install Homebrew, Python, Ollama, Pango, pandoc, Qdrant, and git. Each `brew install` is a no-op for anything you already have, so the block is safe to run as-is on a fully-configured machine or a freshly-wiped one.

Run it in three parts. Two of them need a key-press: **Part 0** (Homebrew) triggers the macOS *"install command line developer tools"* dialog, and **Part 1** (`brew install`) asks `[y/n]`. The big paste — **Parts 2–5** — has no prompts, so it runs unattended. Pasting a `brew install` *inside* a block doesn't work: brew stops to ask `[y/n]` and reads the lines pasted behind it as (rejected) answers, so the block must keep that one line separate.

Part 0 — Homebrew (interactive on a bare-metal Mac). Run alone; on a wiped machine it pops the *"command line developer tools"* dialog — click **Install**, wait for it to finish, then confirm `brew` answers before continuing. (Already have Homebrew? Skip to Part 1.)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.s
brew --version    # must print a version before you proceed
```

Part 1 — the toolchain (interactive: press `y`). Run this line on its own; when brew asks `Do you want to proceed with the installation? [y/n]`, type `y` and **Enter**. (Each formula is a no-op if already present.)

```
brew install python@3.13 ollama pango pandoc git
```

Parts 2–5 — everything else, one uninterrupted paste (no prompts):

```
# 2. Start Ollama + put Homebrew's keg-only python3 on PATH (else python3 stays at the syste
brew services start ollama
PY_LINE='export PATH="/opt/homebrew/opt/python@3.13/libexec/bin:$PATH"'
grep -qxF "$PY_LINE" ~/.zprofile 2>/dev/null || echo "$PY_LINE" >> ~/.zprofile
eval "$PY_LINE"

# 3. Models — embedding (required) + the default language model (small, fast, reliably cited
ollama pull nomic-embed-text
ollama pull llama3.1:8b         # the default; add `ollama pull gemma3:27b` on 32GB+ for the

# 4. Qdrant binary (needs ~/bin on PATH; the grep makes the line idempotent)
mkdir -p ~/qdrant_storage ~/bin
curl -L https://github.com/qdrant/qdrant/releases/latest/download/qdrant-aarch64-apple-darwi
mv qdrant ~/bin/qdrant
grep -q 'export PATH="$HOME/bin' ~/.zshrc 2>/dev/null || echo 'export PATH="$HOME/bin:$PATH"

# 5. Clone, install, launch
mkdir -p ~/Developer && cd ~/Developer
git clone https://github.com/mbarberony/AIStudio.git
cd AIStudio
./ais_install
source ~/.zshrc
ais_start
```

First run auto-indexes the **demo** and **help** corpora (~60s) and opens the UI on `llama3.1:8b`. First query is slow (model load, 20–50s); after that ~6–7s. `ais_help` lists commands; `ais_stop` when done.

> *In the UI sidebar the demo corpus defaults to **Top K = 10**. Full knobs in Annex 3.*

Annex 1 — Tools Used in This Guide

Tool	What it does	URL
Homebrew	Package manager for macOS	https://brew.sh
Python	Programming language AIStudio runs on	https://www.python.org
Ollama	Local AI model runtime	https://ollama.com
Pango	Text-rendering library for PDF benchmark reports	https://pango.gnome.org
pandoc	Document converter (Markdown → HTML) for PDF benchmark reports	https://pandoc.org
Qdrant	Vector database for document storage	https://qdrant.tech
GitHub / git	Code hosting and version control	https://github.com

The language models themselves — Google **Gemma**, Meta **Llama**, Mistral AI's **Mistral** — are downloaded through Ollama in Step 4.

Annex 2 — Troubleshooting

Entries follow the order of the steps, so you can scan to where you are.

Something unexpected happened — close and reopen Terminal (**⌘ Space**, type **Terminal**, **Enter**), then re-run the last command. A fresh terminal always starts with a clean environment.

Any command errors with `getcwd: cannot access parent directories` or `The current working directory must exist to run …` — your Terminal is sitting *inside a folder that was deleted, moved, or renamed* (common right after removing `~/Developer/AIStudio`). The shell can't run anything until it's in a folder that exists. Fix:

```
cd ~
```

Then re-run your command. (Opening a brand-new Terminal window does the same thing — new windows always open in your home folder.)

The single most common cause of "it installed but the command isn't found": you're on bash, not zsh. If your Terminal prompt ends in `$` (e.g. `Mac:~ yourname$`) instead of `%`, your

account uses the older bash shell, and this guide's PATH/alias lines (which go into `~/.zprofile` and `~/.zshrc`) are being ignored. Symptoms: `brew`, `qdrant`, or any `ais_*` command reports `command not found` right after you installed it. **Fix:** `chsh -s /bin/zsh`, then **⌘ Q** to quit Terminal, reopen it, and re-run the step you were on. See Step 0. This one switch resolves most of the entries below at once.

(TL;DR) `brew install` prints `Invalid input. Please press 'y'…` over and over, then `y: command not found` — you pasted a block that contained `brew install`. brew stopped to ask `[y/n]` and read every following pasted line as a (rejected) answer; by the time you typed `y`, brew had already aborted. Run `brew install …` on its own line, press **`y`**, let it finish, *then* paste the rest. This is why the TL;DR keeps Part 1 (`brew install`) separate from the Parts 2–5 paste.

(Step 2/3) `brew install` seems to hang doing nothing — it's waiting for you. brew asks `Do you want to proceed with the installation? [y/n]` and pauses; type **`y`** and **Enter**.

(Step 1) `brew --version` returns `command not found` — Homebrew's PATH isn't loaded in this Terminal session. On a modern Mac, opening a *new* Terminal window fixes it (the installer registers Homebrew system-wide). To load it into the current window without reopening, run `eval "$(/opt/homebrew/bin/brew shellenv)"`. On older Macs, run the three `~/.zprofile` lines at the end of Step 1. (Note: the command is `brew --version` — two dashes. `brew -version` with one dash prints Homebrew's help and an "Unknown command" error.)

(Step 2) `python3 --version` still shows `3.9.x` after `brew install python@3.13` — this is expected, not a failure. Homebrew installs `python@3.13` "keg-only" — it does not replace the system `python3` on your PATH. Run the PATH block in Step 2 (the `PY_LINE=…` lines), then open a new Terminal window if it still reports 3.9. AIStudio needs 3.10 or newer.

(Step 2) `python3` not found at all — Python isn't installed; run `brew install python@3.13`, then the Step 2 PATH block.

(Step 3) `ollama list` hangs or errors — Ollama isn't running. If brew-installed: `brew services start ollama`. Otherwise: `ollama serve` in a separate tab. **Do not run `ollama serve` if `ollama list` already worked** — Ollama is already running as a background service, and a second copy fails with `address already in use` (harmless, but not needed).

(Step 3/10) UI shows "Ollama not running" — run `brew services start ollama`, then `ais_start` again.

(Step 5) `grep: /Users/<you>/.zshrc: No such file or directory` — harmless. On a brand-new account `~/.zshrc` doesn't exist yet; the command still creates it and adds the PATH line. You can ignore this message and continue.

(Step 5) `qdrant` — `command not found` — `~/bin` isn't on your PATH. Run `source ~/.zshrc` (and confirm the PATH line is in `~/.zshrc`, per Step 5). If you're on bash, see the bash/zsh note at the top of this annex.

(Step 7) `cd: AIStudiocd: No such file or directory`**, or** `./ais_install: No such file or directory` — two commands got run on one line, or you're not inside the repo folder. Run `cd ~/Developer/AIStudio` on its own line, confirm your prompt ends in `AIStudio %`, then run `./ais_install`.

(Step 8) `ais_start` **(or any** `ais_*`**) returns** `command not found` **immediately after** `ais_install` **succeeded** — the aliases were written to `~/.zshrc` but this Terminal session hasn't loaded them yet. Run `source ~/.zshrc` (exactly as the install summary says), then the command. If it *still* fails, you're on bash — see the bash/zsh note at the top of this annex. Note: `source ~/Developer/AIStudio/.venv/bin/activate` activates the Python environment but does **not** load the `ais_*` aliases — that's `source ~/.zshrc`.

(Step 9) `(.venv)` **not in your prompt** — run `source ~/Developer/AIStudio/.venv/bin/activate`.

(Step 10) `curl …/health` **doesn't return** `{"status":"ok"}`**, or UI shows "Error loading corpora"** — the browser opened before the backend finished starting. Hard-refresh (`Cmd+Shift+R`). If it persists, run `ais_stop` then `ais_start`.

(Step 11) Stats show 0 chunks — the corpus isn't ingested yet. Use the UI **Add** button to upload and ingest documents.

Annex 3 — Tuning Parameters

The settings sidebar (Step 11) controls retrieval and generation. Defaults are tuned for the demo corpus; adjust per corpus as needed.

Parameter	Default	Effect
Model	`llama3.1:8b`	The language model that writes answers. Small, fast, and reliably cited for the demo; switch to `gemma3:27b` for the SEC 10-K corpus.
Top K	10	Chunks retrieved per query. Higher = more context, slightly slower. **10 is the default** — the demo has small documents (as few as 20 chunks) that only surface reliably at K=10, and the SEC corpus needs K=10 for cross-firm multi-source queries.
Temperature	0.3	LLM creativity. Lower = more factual. Keep at 0.3 for document Q&A.
Retrieval Mix	0.5	Blends keyword matching with semantic understanding. Drag left toward **Literal** (exact word matching) or right toward **Conceptual** (related meaning even when exact terms differ).

Parameter	Default	Effect
		Center (0.5) works well for most queries; try full Conceptual for thematic questions, center-to-Literal for specific entity or term lookups.
Relevance Threshold	0.3–0.5	Filters out retrieved chunks that scored too low to be useful. Low-score chunks cause hedged or incorrect answers. Set lower (0.3) for corpora with small documents; higher (0.5) for large uniform corpora like SEC 10-K. Configured per-corpus — the demo uses 0.3, sec_10k uses 0.5.
Timeout (s)	300	How long AIStudio waits for the model before giving up. The default is deliberately generous — it covers slow, large models (a 70B answer can take ~140–170s; cutting it off mid-generation wastes the whole run). Raise it only if you switch to a very large model and hit timeouts; there's rarely a reason to lower it. Configured per-corpus, overridable per-query in the left panel.

Each of Top K, Relevance Threshold, and Timeout resolves the same way: the value in the left panel wins; leave it blank and AIStudio uses the corpus's saved default; if the corpus has none, it falls back to the system default. So you can tune once per corpus (in **Edit Corpus**) or ad-hoc per query.

> *For more on query settings, see HOWTO.md.*

For architecture context, see docs/architecture_decisions.md. For day-to-day usage, see HOWTO.md. For guided walkthroughs, see TUTORIAL.md.